

ACK Freight App — System Overview

How the website and mobile apps are built, what services they depend on, and how to ship changes. Written for a new team member. Last updated: October 2, 2026.

1. What this system is

ACK Freight is a freight dispatch and trip-logging platform used by three kinds of people. It runs as a **website** and as **iOS and Android apps**, all built from **one codebase** (React Native with Expo) and sharing **one backend** (Firebase).

Role	What they do
Customer	Submits trip requests (route, date, pick-up time, AWBs with weight/type/priority, documents, optional note). Tracks status and sees the driver's live location once a trip starts. Cannot edit a request after a driver starts it.
Admin / Dispatch	Reviews requests, assigns a driver to each AWB, corrects weights/priority, sees live maps, sends invoices through QuickBooks, reviews the Drivers Log, payroll, driver records, deleted trips, and manages user accounts.
Driver	Sees assigned trips, taps Start, shares live location while on a trip, completes the AWBs they actually delivered (with ULD numbers, proof photos, optional note), and can log trips directly.

A user's role is stored on their profile. At sign-up, an email on the admin list (`EXPO_PUBLIC_ADMIN_EMAILS` in the environment config) becomes an admin; everyone else becomes a driver or customer depending on which sign-up link they used. Admins can change roles later in Manage Users.

2. Architecture at a glance

```
YOUR COMPUTER (code)  ---->  GitHub (backup + history)
    |
    +--> Expo web export ----> Firebase Hosting ---> ack-freight.web.app  (website)
    +--> firebase deploy ----> Cloud Functions (Cloud Run)   (backend logic)
    +--> eas update  -----> Expo Update service ---> installed phones (JS-only changes)
    +--> eas build / submit --> TestFlight (iOS) /  APK link (Android)

Website + iOS app + Android app
    | (all talk to the same backend at runtime)
    v
  Firebase: Authentication | Firestore (data) | Storage (files) | Cloud Functions
    |
    v
Outside services: QuickBooks Online (invoices) | Gmail (emails) | Expo push (notifications) | Google Maps
```

Key idea: the website and apps are just different front ends over the same Firebase project. Anything an admin changes on the website is immediately visible in the apps because they read the same Firestore database.

3. Technology stack

Layer	Technology
Front end (web + native)	React Native 0.86, Expo SDK 57, TypeScript, React Navigation, react-native-web for the website
Backend	Firebase Cloud Functions (v2, Node.js 20) written in JavaScript in <code>functions/index.js</code>
Database	Cloud Firestore with security rules (<code>firestore.rules</code>)
File storage	Firebase Storage with rules (<code>storage.rules</code>)
Auth	Firebase Authentication (email + password)
Maps / tracking	<code>react-native-maps</code> + <code>expo-location</code> on native; an embedded Google Maps iframe on web
Builds / OTA	Expo Application Services (EAS Build, Submit, Update)
CI	GitHub Actions workflow that runs the TypeScript type-check on every push

4. Services and accounts

These are the external accounts the system depends on. A new team member needs access to the ones relevant to their work.

Service	What it's used for	Where it's managed
Firebase (project <code>ack-freight</code>)	Hosting, Authentication, Firestore, Storage, Cloud Functions, Cloud Scheduler	<code>console.firebase.google.com</code>
Google Cloud (same project)	Functions actually run on Cloud Run; IAM permissions; Secret Manager (credentials); Logs Explorer; billing; API keys	<code>console.cloud.google.com</code>
Expo / EAS (account <code>victor.abreu13</code> , project <code>ack-freight-driver-log</code>)	Cloud builds, App Store submission, over-the-air updates, signing credentials, per-environment variables (production, preview), push notification service	<code>expo.dev</code>
Apple Developer + App Store Connect	iOS signing certificate and provisioning profile, TestFlight distribution to testers. Bundle ID <code>com.ackfreight.driverlog</code> . Individual developer account.	<code>developer.apple.com</code> , <code>appstoreconnect.apple.com</code>
GitHub (<code>victorabreu13/ackfreightapp</code>)	Source code, version history, Issues, type-check workflow	<code>github.com</code>
Intuit / QuickBooks Online	Creates and emails customer invoices. A production developer app is connected to the company ACK TEAM LLC via OAuth.	<code>developer.intuit.com</code> + QuickBooks company
Gmail	Sends the system's notification emails from the Cloud Functions (SMTP with an app password)	Google account settings
Google Maps Platform	Android map key (in <code>app.json</code>) for live tracking	Google Cloud Console → APIs & Services
OneDrive	Where the project folder lives on the developer's machine (see Troubleshooting for the side effects)	—

Not used: Google Play. Android is installed from a direct APK link; production `.aab` files have been built but never submitted to Play.

Secrets: no passwords or API secrets are stored in the code. Gmail and QuickBooks credentials live in Google Secret Manager; the Firebase client settings live in a local `.env` file (git-ignored) and in EAS environment variables for builds.

5. Code layout

Path	What's in it
------	--------------

<code>App.tsx</code>	App entry: auth provider, navigation container, header, auto-update components, web URL routes
<code>src/screens/</code>	One file per screen (Login, Dispatch, DriversLog, ManageUsers, DriverPayroll, DeletedTrips, DriversAvailable, etc.)
<code>src/components/</code>	Shared UI: TripRequestForm, TripCard, DateField, AwbDriverPicker, LiveMap (native) and LiveMap.web, AppHeader, AutoReload (web), NativeAutoUpdate (native)
<code>src/services/</code>	All Firestore/Cloud Function calls from the front end (trips, tripRequests, users, quickbooks, storage, notifications, customers)
<code>src/navigation/</code>	RootNavigator picks the screen set by role (admin, customer, driver) and a navigation ref used by the header logo
<code>src/types/index.ts</code>	Shared data types and helpers, including the status roll-up logic
<code>src/utils/</code>	Cross-platform alerts and the local-date helper
<code>functions/index.js</code>	Every Cloud Function (backend)
<code>firestore.rules</code> , <code>storage.rules</code>	Database and file access rules
<code>scripts/deploy-web.js</code>	One-command web build + deploy (also stamps a version file)
<code>legal/</code>	Terms and Privacy pages copied into the web build
<code>app.json</code> , <code>eas.json</code>	Expo app config (permissions, runtime version, update URL) and EAS build profiles/channels

6. Data model

Firestore collections

Collection	Holds	Who can write
users	Profile per account: name, email, role, pay/bill rates, push token, QuickBooks customer id, active flag	Owner creates their own; admin edits name and rates directly; role, email, password, active and delete go through Cloud Functions
trips	The Drivers Log: one entry per completed delivery (date, times, route, AWBs, ULD numbers, kilograms, proof files, paid, invoiced)	Drivers create their own; admin can only toggle paid/invoiced; deletion only via a function
tripRequests	A customer's request with up to 10 AWB lines. Each AWB line has its own driver, status, weight, priority and file links. Also holds status, note, and per-driver live locations.	Customer while not yet started; admin always; drivers only through start/complete functions
deletedTrips	Audit copy of every deleted trip so an admin can restore it	Functions only
quickbooksAuth	QuickBooks OAuth tokens	Functions only. Nobody, including admins, can read it from the app.

Storage paths

- proofs/{driverId}/... — proof-of-freight photos and documents; the driver writes, the driver and admins read.
- tripRequestDocs/{customerId}/{requestId}/... — AWB, authorization and delivery-order files; the customer and admins write; assigned drivers can read.

Status logic

Every AWB line moves through **Unassigned** → **Assigned** → **In Progress** → **Completed**. The trip request's overall status is calculated from its lines: it is Completed only when every line is Completed, In Progress as soon as any line has started. An admin then sets it to **Invoiced** (by QuickBooks invoice or manually); **Cancelled** is admin/customer only. A line's priority (Low / Normal / High) is informational.

7. Main workflow: from request to invoice

1. **Customer submits** a trip request on the web or app. Files upload to Storage. A function emails admins and sends them a push notification.
2. **Admin assigns a driver to each AWB** in Dispatch. The assigned driver is emailed and gets a push notification plus an app-icon badge.
3. **Driver taps Start.** The `startMyAwbLines` function marks their AWBs In Progress and stamps the start time. The customer can no longer edit the request.
4. **Live tracking.** While any of the driver's AWBs is in progress, the app sends the phone's location every ~25 seconds (`updateMyLiveLocation`). Customer and admin screens subscribe to the request and show the position on a map. Location is only the current position, with no history.
5. **Driver completes.** They tick which of their in-progress AWBs they actually delivered, enter ULD numbers, attach proof, optionally add a note, and submit. A Drivers Log entry is created and `completeMyAwbLines` marks those AWBs Completed. AWBs left unchecked stay In Progress for a later separate run. When all AWBs are done the request becomes Completed and admins and the customer are emailed.
6. **Admin invoices.** After correcting weights if needed, the admin clicks Send Invoice. `sendQuickBooksInvoice` finds or creates the QuickBooks customer, creates a non-taxable invoice (per trip or per kilogram, using the customer's bill rate), and marks the request Invoiced. Trips from that request then show an Invoiced badge in the Drivers Log.
7. **Payroll.** Driver Payroll totals what each driver is owed by period (per trip or per kg rate) and lets admins mark trips paid.

Known limitation: QuickBooks' "email the invoice" call fails with a server error on Intuit's side every time. The invoice is still created correctly; the admin sends the email manually from inside QuickBooks. This has been investigated and is not caused by this app.

8. Cloud Functions reference

Function	Type	Purpose
<code>onTripRequestCreated</code>	Firestore trigger	Email + push to admins for a new request
<code>onTripRequestAssigned</code>	Firestore trigger	Email + push to a newly assigned driver (only their AWBs)
<code>onTripRequestStatusEmails</code>	Firestore trigger	Completed email to admins and the customer
<code>sendDailyTripSummary</code>	Scheduled, 11:59 pm New York time	Daily trip summary email to admins
<code>startMyAwbLines</code> / <code>completeMyAwbLines</code>	Callable	Driver moves their own AWBs forward; complete takes the chosen AWB indexes
<code>updateMyLiveLocation</code>	Callable	Stores a driver's current position on the request
<code>checkDuplicateUld</code>	Callable	Warns if a ULD number was already logged under the same AWB
<code>deleteTrip</code> , <code>listDeletedTrips</code> , <code>restoreDeletedTrip</code>	Callable	Audited deletion and admin recovery of trips

createCustomer	Callable	Admin creates a customer account on their behalf
setUserRole, updateUserEmail, updateUserPassword, setUserActive, deleteUserAccount	Callable (admin only)	Account management that touches the Firebase login itself. Deactivating blocks sign-in but keeps history; delete is permanent.
startQuickBooksAuth, quickbooksOAuthCallback, getQuickBooksStatus, disconnectQuickBooks	Callable / HTTP	Connect and manage the QuickBooks link (OAuth with a one-time state check)
sendQuickBooksInvoice	Callable (admin only)	Creates the invoice and marks the request invoiced

All "admin only" functions verify the caller's role from Firestore before doing anything. Notification emails skip one excluded personal address by design.

9. How to ship a change

Pick the row that matches what changed. The website always updates with every web deploy; the question is how the phone apps get it.

What changed	Commands	Result
Anything visible on the website	<code>npm run deploy:web</code>	Builds, stamps a version, uploads to Hosting. Open browser tabs reload themselves within about a minute.
Backend function code	<code>firebase deploy --only functions:NAME</code>	Live in minutes. A brand-new function needs a one-time manual permission fix (Section 10).
Database or file rules	<code>firebase deploy --only firestore:rules</code> (or <code>storage</code>)	Live immediately
JS/React-only change for the phone apps (screens, filters, fields)	<code>eas update --branch production --message "... " --environment production --non-interactive</code>	Installed apps download it and apply it on the next open. No rebuild, no TestFlight wait.
Native change (new native library, permission, app.json native config, icon)	<code>eas build --profile production --platform all</code> then <code>eas submit --profile production --platform ios --latest</code>	New TestFlight build (Apple processing 5–10 min). Android production output is an <code>.aab</code> that cannot be installed directly.
A tap-to-install Android link for testers	<code>eas build --profile preview --platform android</code>	APK link; install steps in Section 10

Routine after every change

1. Run `npx tsc --noEmit`; it must pass with no errors.
2. Deploy the appropriate part(s) from the table above.
3. Commit and push to GitHub (`git add -A`, commit with a clear message, `git push origin master`). The project convention is to push after every change.

Over-the-air updates are safe by design. The apps use a "fingerprint" runtime version: an update is only delivered to app builds with matching native code, so a mismatch fails safely instead of crashing. Still, an OTA update reaches every driver on their next app open with no review step, so test on the website first when possible.

10. What depends on what

If this is down or wrong...	...this breaks
Firebase Authentication	Nobody can log in on any platform
Firestore / rules	All data in every screen; a bad rules deploy can block saves with permission errors
Cloud Functions	Driver start/complete, live location, invoicing, emails, user management, trip deletion. Customers can still submit requests and admins can still assign.

Secret Manager entries	Emails (Gmail) and invoicing (QuickBooks) stop working
QuickBooks connection (token expired or disconnected)	Invoicing only; reconnect from the Dispatch screen
Expo EAS Update	Phones stop getting OTA changes; the website is unaffected
Apple certificate / profile (expire Sept 2027)	New iOS builds can't be signed
TestFlight build older than 90 days	That build stops opening for testers
Cloud Functions Node 20 runtime (end of life Oct 30, 2026)	Function deploys will be refused until the runtime is upgraded

11. Troubleshooting and known quirks

Symptom	Cause and fix
New function deploy fails with "Failed to set the IAM Policy on the Service"	Happens on the first deploy of every new function. In Google Cloud Console open Cloud Run → the service (lowercase function name) → Security → Allow public access , save, then redeploy that function. The app still requires login inside the function.
Calling a new function returns <code>internal</code>	Same cause: the permission step was never completed even if the deploy printed success. Apply the fix above.
Firebase CLI says "credentials are no longer valid" (or a confusing "Assertion failed ... hosting target")	Session expired. Run <code>firebase login --reauth</code> in a terminal.
Function deploy: "User code failed to load ... Timeout"	Transient slowness reading files from OneDrive. Retry the same command.
Web export error "Unable to resolve module ./node_modules/expo/AppEntry", or EAS "Install dependencies" failure	OneDrive corrupts <code>node_modules</code> sometimes. Delete <code>node_modules</code> and <code>package-lock.json</code> , run <code>npm install</code> , retry.
Android install says "App not installed" (Samsung)	Open the link in Chrome; allow Chrome under Install unknown apps; turn off Samsung Auto Blocker; when Google Play Protect warns, tap Install anyway (the small link, not the OK button).
Dates off by one in the evening	Fixed by always using the local-date helper (<code>src/utils/date.ts</code>); never use <code>toISOString()</code> to build a date string.
Browser shows an old version of the website	Hosting caches files up to an hour; open tabs poll a version file and reload automatically, and a hard refresh also works.

Things to know about channels

- Production builds (TestFlight, `.aab`) listen on the **production** update channel; preview APKs listen on the **preview** channel. An `eas update` to one branch does not reach apps on the other, so publish to both if testers use preview APKs.
- Apps built before OTA was added (early preview APKs) have no update client and must be reinstalled.
- EAS builds beyond the monthly included credits are billed pay-as-you-go; OTA updates are the cheap path for JS-only changes.

12. Conventions

- Roles and permissions are enforced twice: hidden in the UI and again by Firestore/Storage rules and function checks. Never rely on the UI alone.
- Trips and trip requests copy the driver/customer name and email at creation, so renaming or deleting an account does not rewrite history.
- Dates are stored as local calendar-day strings (`YYYY-MM-DD`); the business time zone is `America/New_York`.
- Some screens are web-only on purpose (Drivers Record, Driver Payroll, Deleted Trips, Drivers Available) to keep the phone menu light.

- Deactivate an account (keeps history) instead of deleting it; deactivated drivers disappear from assignment pickers but still appear in payroll and records.

13. Setup checklist for a new teammate

1. Get access to: GitHub repo, Firebase/Google Cloud project `ack-freight`, Expo account/project, and (for iOS) the Apple Developer / App Store Connect team.
2. Install Node, Git, and run `npm install -g firebase-tools eas-cli`; then `firebase login` and `eas login`.
3. Clone the repo, run `npm install`, and get the local `.env` file (Firebase client settings and the admin email list) from the project owner. It is not in GitHub.
4. Confirm the toolchain with `npx tsc --noEmit`, and preview locally with `npx expo start --web`.
5. Read Sections 9–11 before the first deploy.

Maintenance reminders: upgrade the Cloud Functions runtime before October 30, 2026; refresh iOS builds before the 90-day TestFlight limit; Apple certificate and profile expire in September 2027.